

■# TNT AI/ML Testing and Validation Report

Project: TNT (Tap N Take) Campus Food and Stationery Ordering Platform

Repository reviewed: C:\TNT_MAIN_PROJECT-main

Report date: 29 July 2026

Previous report compared: C:\Users\jatin\Downloads\TNT_Testing_Validation_Report (1).pdf, compiled 28 July 2026

Scope: AI/ML implementation in tnt-backend-main/app/ml and tnt-backend-main/app/modules/ai_intelligence

1. Executive Summary

The AI/ML implementation has moved beyond the previous report's description of "AI Services (heuristic)" by adding a production ML layer with model registry, model artifacts, training pipelines, prediction services, backtesting, drift detection, shadow logging, scheduled retraining, and promotion/rollback controls.

The current validation result is mixed but materially improved:

Area	Current Result	Verdict
AI planner functional tests	88 passed / 0 failed	PASS
ML bridge tests	6 passed / 0 failed	PASS
ML engine tests	60 passed / 2 skipped / 0 failed	PASS
Targeted AI/ML coverage run	154 passed / 2 skipped	PASS
Combined coverage for app.ml + app.modules.ai_intelligence under targeted run	49%	NEEDS IMPROVEMENT
Core planner coverage	93-99% for demand, ETA, reorder, slot, vendor ranking	STRONG
ML model confidence	Low for all trained models	NEEDS DATA

The major improvement over the prior report is test coverage for the planner layer: the earlier PDF identified demand, ETA, reorder, slot, and vendor ranking modules at roughly 19-30% coverage. The current targeted run shows the core planner modules at 93-99% coverage. However, several surrounding AI services remain weakly tested, especially production upgrade automation, Redis AI cache, vendor speed service, enhanced ETA, and top-level service/router layers.

Model performance metrics should not be interpreted as production-quality accuracy yet. The stored training run reports perfect regression metrics, but the datasets are extremely small: 4-28 rows for the regression models and 27 rows for fraud detection. This is overfitting, not proof of generalisation. The correct operational posture is to keep the heuristic and safety gates as the primary serving path until live data volume improves.

2. Testing Methodology

Validation used four evidence streams:

- 1. **Previous-report comparison:** Extracted and reviewed the 28 July 2026 validation PDF. That report marked the broader platform production-ready but explicitly called the AI/ML planner layer the weakest-tested area and stated that AI features were heuristic rather than trained-model driven.
- 2. **Code review:** Reviewed the implemented ML architecture under app/ml and AI intelligence modules under app/modules/ai_intelligence.
- 3. **Automated functional tests:** Executed ML and planner pytest suites from the backend virtual environment.

4. **Targeted coverage instrumentation:** Ran coverage only for `app.ml` and `app.modules.ai_intelligence` to avoid confusing AI/ML validation with unrelated app-wide coverage gates.

Commands Executed

```
.\.venv\Scripts\python.exe -m pytest test_ml_bridge.py -q --no-cov
.\.venv\Scripts\python.exe -m pytest test_ai_planners.py -q --no-cov
.\.venv\Scripts\python.exe -m pytest tests\test_ml_engine.py -q --no-cov
.\.venv\Scripts\python.exe -m pytest `
  tests\test_ml_engine.py test_ai_planners.py test_ml_bridge.py `
  --cov=app.ml --cov=app.modules.ai_intelligence `
  --cov-report=term --cov-fail-under=0 -q -o addopts=""
```

Execution Notes

- The first sandboxed run of `tests/test_ml_engine.py` produced three failures caused by sklearn parallel tuning hitting `WinError 5: Access is denied`.
- The same ML engine suite passed when rerun outside the sandbox, which indicates the failures were environment permission related rather than application logic failures.
- Warnings remain non-blocking: Starlette `ht t p x` deprecation, sklearn tuning-space warning, and one XGBoost fit warning on tiny data.

3. Automated Test Results

Test Suite	Purpose	Result	Runtime
<code>test_ai_planners.py</code>	ETA, demand, reorder, slot, vendor ranking, signals, hard safety rules, model fallback paths	88 passed	12.89s
<code>test_ml_bridge.py</code>	Model availability, missing model fallback, prediction exceptions, NaN/missing feature guard, confidence threshold fallback	6 passed	3.58s
<code>tests/test_ml_engine.py</code>	Registry, features, training, predictions, explainability, router, CV tuning, temporal split, backtesting, shadow logging, drift, retraining, promotion, rollback	60 passed, 2 skipped	246.40s
Combined targeted AI/ML coverage run	Same three suites with coverage against AI/ML packages only	154 passed, 2 skipped	286.63s

Coverage Summary

Package / Module Area	Coverage	Assessment
Total targeted AI/ML packages	49%	Improved but incomplete
<code>app.ml.dataset_builder</code>	90%	Strong
<code>app.ml.features</code>	84%	Strong
<code>app.ml.backtest</code>	87%	Strong
<code>app.ml.drift</code>	86%	Strong
<code>app.ml.registry</code>	79%	Moderate
<code>app.ml.retraining</code>	73%	Moderate
<code>app.ml.predictions</code>	66%	Moderate
<code>app.ml.promotion</code>	67%	Moderate
<code>app.ml.training_pipeline</code>	47%	Weak
<code>app.ml.router</code>	40%	Weak

Package / Module Area	Coverage	Assessment
ai_intelligence.planners.eta_engine	99%	Strong
ai_intelligence.planners.demand_planner	93%	Strong
ai_intelligence.planners.reorder_engine	96%	Strong
ai_intelligence.planners.slot_planner	95%	Strong
ai_intelligence.planners.vendor_ranker	97%	Strong
ai_intelligence.ml_bridge	73%	Moderate
ai_intelligence.signals	98%	Strong
ai_intelligence.utils.scoring	98%	Strong
ai_intelligence.analytics_service	9%	Critical gap
ai_intelligence.enhanced_eta_engine	12%	Critical gap
ai_intelligence.production_upgrades	0%	Critical gap
ai_intelligence.redis_ai_cache	0%	Critical gap
ai_intelligence.service	17%	Critical gap
ai_intelligence.vendor_speed_service	14%	Critical gap

4. Module Validation Details

4.1 ETA Prediction

Purpose: Predict order preparation or pickup ETA using vendor, queue, slot occupancy, item count, time-of-day, weekday, and rush-hour features.

Implementation reviewed:

- Heuristic planner: app/modules/ai_intelligence/planners/eta_engine.py
- ML prediction service: app/ml/predictions.py
- Training pipeline: app/ml/training_pipeline.py
- Backtesting: app/ml/backtest.py
- Shadow logging: app/modules/ai_intelligence/ml_bridge.py

Testing methodology:

- Unit tests for default response, queue depth, base preparation time, vendor efficiency, delay-risk classification, real-slot prediction, ML success path, model exception fallback, and shadow logging.
- ML engine tests for feature extraction, training, temporal split, registry metrics, explainability, and backtesting.

Current validation result:

- Planner coverage: 99%.
- Training artifact exists: eta_prediction_v2.
- Functional tests pass.

- Shadow logging is verified for model-vs-heuristic comparison.

Training metrics from stored baseline run:

Metric	Value
Rows trained	26
Best model	RandomForest
MAE	0.000
RMSE	0.000
R2	1.000
Confidence assessment	Low

Assessment: Functionally implemented and now well covered at planner level, but model accuracy is not reliable because only 26 training rows were available. Perfect scores indicate memorisation.

Comparison with previous report: Improved. The previous report described ETA as implemented but heuristic and under-tested. The current implementation includes ML model artifacts, registry integration, fallback handling, temporal split tests, backtesting, and strong planner coverage.

4.2 Demand Forecasting

Purpose: Forecast future order demand using hour, weekday, day-of-month, month, and rush-hour features.

Implementation reviewed:

- Heuristic planner: `app/modules/ai_intelligence/planners/demand_planner.py`
- Training pipeline: `app/ml/training_pipeline.py`
- Dataset builder: `app/ml/dataset_builder.py`
- Prediction service: `app/ml/predictions.py`

Testing methodology:

- Tests cover no-data paths, data-backed demand planning, insufficient-history fallback, model success path, model exception fallback, volatility calculation, and capacity-gap recommendations.
- ML engine tests validate feature extraction and training workflow.

Current validation result:

- Planner coverage: 93%.
- Training artifact exists: `demand_forecast_v2`.
- Functional tests pass.

Training metrics from stored baseline run:

Metric	Value
Rows trained	28
Best model	XGBoost
MAE	0.000
RMSE	0.000
R2	1.000
Confidence assessment	Low

Assessment: The implementation is structurally complete and validated through tests, but model output should remain secondary to heuristic planning until longer vendor histories are available.

Comparison with previous report: Improved in automated coverage and model infrastructure. The previous report identified demand planning as a weak AI/ML coverage area and noted a daily forecast SQLite date bug. Current tests include the demand planner and ML fallback paths.

4.3 Slot Recommendation

Purpose: Recommend slots and capacity adjustments based on occupancy, hour, weekday, rush hour, average completion time, and max capacity.

Implementation reviewed:

- Planner: `app/modules/ai_intelligence/planners/slot_planner.py`
- Prediction service: `app/ml/predictions.py`
- Training pipeline: `app/ml/training_pipeline.py`

Testing methodology:

- Tests cover off-peak and peak scoring, full-slot handling, zero-capacity protection, capacity recommendations, model success and fallback paths, and hard exclusion of slots at or above 90% capacity.

Current validation result:

- Planner coverage: 95%.
- Training artifact exists: `slot_recommendation_v2`.
- Functional tests pass.
- Safety rules remain active regardless of model output.

Training metrics from stored baseline run:

Metric	Value
Rows trained	5
Best model	RandomForest
MAE	0.000
RMSE	0.000
R2	1.000
Confidence assessment	Low

Assessment: The rule layer is robust and well tested. The ML model is not yet statistically meaningful because it trained on only 5 slots.

Comparison with previous report: Improved. Previous AI slot recommendation was effectively heuristic and under-tested. Current tests verify both model bridge behavior and safety-rule enforcement.

4.4 Vendor Ranking

Purpose: Rank vendors using completion rate, average rating, repeat customer rate, cancellations, refunds, total orders, live load, and express pickup eligibility.

Implementation reviewed:

- Planner: `app/modules/ai_intelligence/planners/vendor_ranker.py`
- Recommendation integration: `app/modules/recommendations/ranking_service.py`

- Training pipeline: `app/ml/training_pipeline.py`
- Backtesting: `app/ml/backtest.py`

Testing methodology:

- Tests cover no-vendor paths, approved vendor ranking, missing slot data, live-load indicator, reasoning labels, satisfaction score fallback, efficiency fallback, ML model path, heuristic fallback, express-pickup hard rule, and source-field transparency.

Current validation result:

- Planner coverage: 97%.
- Training artifact exists: `vendor_ranking_v2`.
- Functional tests pass.

Training metrics from stored baseline run:

Metric	Value
Rows trained	4
Best model	RandomForest
MAE	0.000
RMSE	0.000
R2	1.000
Confidence assessment	Low

Assessment: Ranking logic and guardrails are well tested, but the trained model is not reliable with only 4 vendor rows. The heuristic score remains more trustworthy for production decisions.

Comparison with previous report: Improved coverage and bridge validation. The previous report noted vendor ranking as a weak-tested heuristic service; current tests validate model path, fallback path, and business-rule independence.

4.5 Fraud Detection

Purpose: Detect suspicious order/user patterns using cancellation behavior, order frequency, payment failures, high order amount, device token presence, and historical fraud flags.

Implementation reviewed:

- Rule-based service: `app/modules/fraud/fraud_rules.py`,
`app/modules/fraud/fraud_detection_service.py`
- ML classifier: `app/ml/training_pipeline.py`
- Promotion/rollback controls: `app/ml/promotion.py`

Testing methodology:

- Existing tests cover fraud heuristic fallback, feature extraction, classification evaluation, training metrics, promotion and automatic rollback behavior.

Current validation result:

- Training artifact exists: `fraud_detection_v2`.
- ML engine tests pass.
- Fraud model is still low confidence because current positive fraud labels are insufficient.

Training metrics from stored baseline run:

Metric	Value
Rows trained	27
Best model	RandomForest classifier
Accuracy	0.800
Precision	Not available in stored raw JSON
Recall	Not available in stored raw JSON
F1	Not available in stored raw JSON
Confidence assessment	Low

Assessment: The fraud module should continue to rely on deterministic fraud rules until real positive fraud labels accumulate. The 0.800 accuracy is not enough to validate fraud detection quality because class balance is weak.

Comparison with previous report: Partially improved. The prior report validated admin fraud flags and rule-based fraud behavior. The current implementation adds ML training and rollback controls, but model performance evidence is still not production-grade.

4.6 Recommendation and Personalization

Purpose: Provide personalized item/vendor recommendations and support vendor and slot ranking paths.

Implementation reviewed:

- Recommendation services: `app/modules/recommendations/*`
- Prediction support: `app/ml/predictions.py`
- Preference learning: `app/modules/ai_intelligence/learning/preference_engine.py`

Testing methodology:

- ML engine tests validate personalized recommendation response shape.
- Prior platform report validated user recommendation flow at a functional level.

Current validation result:

- Recommendation behavior is present and tested at a basic integration level.
- Preference engine coverage remains low at 19%.
- Deeper ranking-service and collaborative-filtering behavior should receive additional tests.

Assessment: The module is functional, but validation depth is weaker than the core planners.

Comparison with previous report: Slight improvement for backend ML plumbing, but frontend-level recommendation validation still appears stronger than backend algorithmic validation.

4.7 ML Infrastructure, Registry, Drift, Retraining, and Governance

Purpose: Store model versions, train candidate models, compare metrics, promote better models, roll back degraded models, log shadow predictions, detect drift, and expose ML routes.

Implementation reviewed:

- Registry: `app/ml/registry.py`
- Model metadata table: `app/ml/ml_models_model.py`
- Training: `app/ml/training_pipeline.py`
- Prediction service: `app/ml/predictions.py`

- Drift: `app/ml/drift.py`
- Retraining: `app/ml/retraining.py`
- Promotion: `app/ml/promotion.py`
- Router: `app/ml/router.py`
- Shadow logs: `app/ml/shadow_log_model.py`

Testing methodology:

- Tests cover save/load/versioning, rollback, metric update patterns, model type discovery, temporal split, cross-validation tuning, backtesting, shadow logging, drift reports, scheduled retraining, model promotion, and automatic rollback.

Current validation result:

Component	Coverage	Assessment
Dataset builder	90%	Strong
Registry	79%	Moderate
Backtest	87%	Strong
Drift	86%	Strong
Retraining	73%	Moderate
Promotion	67%	Moderate
Predictions	66%	Moderate
Training pipeline	47%	Needs improvement
ML router	40%	Needs improvement

Assessment: Governance controls are meaningfully implemented and tested, but route-level behavior, failure handling, and full training branch coverage need more work.

Comparison with previous report: Major architectural improvement. The previous report did not evidence a mature ML operations layer. Current code includes versioning, drift reports, retraining logs, model promotion, automatic rollback, and shadow logging.

5. Comparison With Previous Validation Report

Prior Report Finding, 28 July 2026	Current Finding, 29 July 2026	Status
AI/ML planner modules were weakest-tested, roughly 19-30% coverage.	Core planners now show 93-99% coverage in targeted AI/ML run.	Improved
"ML-based predictions" remained a gap; most AI features were heuristic.	Trained model artifacts and registry-backed ML prediction services now exist.	Improved
Prior report claimed broad backend green state: 1105 passing, 0 failed, 16 skipped.	Repo's newer ML baseline doc records 1,167 passed, 18 failed, 16 skipped, 9 errors for <code>pytest -q</code> on 29 July 2026. This report did not rerun the full suite; it reran AI/ML targeted suites.	Discrepancy
Previous AI service validation was largely feature/API focused.	Current validation includes model registry, training, CV, temporal split, backtesting, drift, shadow logging, promotion, rollback, and fallback behavior.	Improved

Prior Report Finding, 28 July 2026	Current Finding, 29 July 2026	Status
AI features were called complete from frontend perspective, but backed by under-tested logic.	Planner logic is now well tested; surrounding services still have low coverage.	Partially improved
No reliable ML accuracy figures were available for production use.	Training metrics are available, but all are low confidence due to tiny datasets.	Improved evidence, not improved model reliability

6. Key Discrepancies and Risks

- Full-suite status conflict:** The previous PDF reports a fully green broad backend suite, while docs/ml_baseline_test_report.md records a later baseline of 1,167 passed, 18 failed, 16 skipped, and 9 errors. The AI/ML targeted suites pass, but the full current repository should not be represented as globally green without another full-suite run.
- Model metrics are misleadingly perfect:** ETA, demand, slot, and vendor ranking all report RMSE 0.000 and R2 1.000. Because training rows range from 4 to 28, these are overfitting indicators.
- Fraud classifier has insufficient labels:** Fraud accuracy is 0.800, but stored raw results do not include precision, recall, or F1, and the dataset lacks real positive fraud examples at scale.
- Service-layer coverage remains weak:** Core planner modules improved dramatically, but the top-level AI service, production upgrade automation, Redis AI cache, enhanced ETA, and vendor speed service remain under-tested.
- Training pipeline still has low branch coverage:** The training pipeline is complex and only 47% covered in the targeted run. Error branches, dependency-missing branches, per-model training variants, and persistence failures should be expanded.
- Environment sensitivity:** Sklearn parallel tuning failed inside the sandbox and passed outside it. CI should explicitly control joblib/sklearn multiprocessing behavior and writable cache paths.

7. Recommendations

- Keep heuristic-first serving until data volume improves.** Maintain current fallback gates for ETA, demand, slot, vendor ranking, and fraud. Do not market the current models as production-accurate ML.
- Set minimum data gates before model promotion.**
 - ETA: at least 200 completed orders with reliable actual_completion_minutes.
 - Demand: at least 90 days of meaningful vendor order history.
 - Slot recommendation: at least 20-50 slots across multiple vendors and load patterns.
 - Vendor ranking: at least 10 approved vendors, preferably many more.
 - Fraud: real positive fraud labels from manual review, chargeback, or admin flag workflows.
- Add precision, recall, and F1 reporting for fraud.** The training output should persist all classifier metrics, not only accuracy.
- Raise service-layer coverage.** Prioritize tests for:
 - app/modules/ai_intelligence/service.py
 - analytics_service.py
 - vendor_speed_service.py
 - redis_ai_cache.py
 - production_upgrades.py

- app/ml/router.py

5. **Add route-level API tests for ML endpoints.** Validate /ml/accuracy, /ml/backtest, /ml/drift, /ml/retrain, /ml/explain, model registry endpoints, and error responses.
6. **Harden CI for ML tests.** Configure a writable pytest cache path and joblib temp folder, and consider forcing n_jobs=1 in CI if Windows multiprocessing remains flaky.
7. **Use live shadow logs to compare model vs heuristic before promotion.** Require backfilled actuals and report: ETA MAE, within-3-minute percentage, within-5-minute percentage, vendor ranking top-1 and top-3 hit rates.
8. **Re-run full-suite validation before release sign-off.** The AI/ML subset is green, but the current repo-level baseline has documented non-ML failures/errors that should be closed or explicitly scoped out.

8. Final Verdict

The AI/ML implementation is functionally validated at the unit and targeted integration level. The strongest areas are the core planner modules, ML bridge fallback behavior, dataset construction, drift checks, and backtesting utilities. Compared with the previous validation report, the AI/ML layer has significantly better automated coverage and a more complete ML operations architecture.

The models themselves are not yet production-trustworthy. Current performance metrics are low-confidence because the training datasets are too small and overfit. The correct production posture is **PASS WITH CONDITIONS**: keep the deterministic heuristics and safety rules as the primary decision path, retain model outputs in guarded or shadow mode, and revisit model promotion after sufficient real operational data is collected.